

Doc. No.: WG21/P0783

Date: 2017-09-11

Authors: Lee Howes lwh@fb.com, Andrii Grynko andrii@fb.com, Jay Feldblum yfeldblum@fb.com

Reply-to: Lee Howes

Email: lwh@fb.com

Audience: SG1

P0783: Continuations without overcomplicating the future

Background

In the Concurrency TS, `std::future` was augmented with support for continuations. However, the feedback leading up to and during the July 2017 meeting in Toronto made clear that the specification for continuation support in the TS was insufficient. The absence of support for executors made the behavior of continuations attached to futures hard to understand and hard to control. LWG2533 expressed concern about where the continuation is run, and other papers including P0667, P0679 and P0701 pointed out more areas of improvement. Much of this feedback relates to the ongoing work on executors that is as yet incomplete but is detailed in P0443. The continuation elements of the Concurrency TS were subsequently not merged into the C++20 standard draft at Toronto.

At Facebook, like at many companies, we have considerable experience using continuations on futures. The open source [Folly](#) library encapsulates a set of primitives widely used within Facebook. Folly Futures supports executors and continuations. From the widespread use of Folly Futures inside Facebook we have learned many lessons. The most important lesson is very similar to that expressed in LWG2533: that it must be specified, and defined precisely, where a continuation is run when attached to a future. In the absence of a strong rule, there is an inherent under-specification or non-determinism:

- If the future is complete at the time of attaching the continuation, is it run in the calling thread-of-execution that attaches the continuation?
- If the future is incomplete at the time of attaching the continuation, is it possible for the continuation to run in the thread-of-execution that completes the future to which the continuation is attached?
- Does the continuation always run in a well-defined thread-of-execution, whatever the completion state is of the future? If yes, to achieve this, the thread-of-execution has to be guaranteed to survive long enough for the continuation to run.

Our experience at Facebook leads us to believe that a known executor should always be available at the point of attaching a continuation to a future, and where the continuation runs should be explicitly defined by the rules of that executor. In this way, and as long as the lifetime of the executor can be guaranteed, the behavior of continuations attached to futures is understandable and controllable. Executor lifetimes have partly been considered by making them value types in P0443. Recent experience in Folly, where they are not value types, validates the design decision to make them value types. (The management of executor lifetimes is out of scope of this paper; that question should be clarified in the ongoing specification of executors.)

Always requiring an executor at the point of continuation attachment, as in calls to the proposed `std::future::then` (as shorthand, `.then`), is possible but clumsy. During discussions in Toronto more than one person expressed a wish

to allow a default executor, which would be used if none was provided to the call to `.then`. If this is to be allowed then there arise further questions:

- Should the default executor be a global default? If so, this will be suboptimal for cases where it makes more sense to allow the continuation to run on the same thread-of-execution as that which completes the future.
- Should an executor be attached to the future or instead to its core (shared state)? If the latter, a continuation attached with `.then` would run on the same thread-of-execution as the completion of the future.

Something close to the second option here is often used in Facebook's libraries. An asynchronous library will typically require an executor to be passed in, and the library will ensure that the future it returns to callers across the library boundary will complete on the caller-provided executor, regardless of whether any further work is to be performed on that caller-provided executor. The asynchronous library will typically attach an empty continuation to the passed executor to make this guarantee. This action has runtime cost, and cognitive load for the user because executor parameters need to be widespread and intrude in places where they are not relevant.

Take a simple example of a library that gets a future and passes it into some other library:

```
void myForwardingFunction() {
    Executor e;
    auto future = LibraryA::getFromSomewhere(e, params);
    LibraryB::sendToSomewhere(future);
}
```

In this case, why did `myForwardingFunction` need to know about an executor at all? It would be difficult to choose an executor here that guarantees forward progress but which does not impose a high cost such as might arise in construction of a thread, ensuring presence of an additional thread pool, etc. In practice, `LibraryB` would use its own internal executor to run the continuation it attaches, but this is not something that `LibraryA` can rely on while providing a safe separation of concerns across the library boundary.

Yet this approach is common to ensure that, in any inter-library interaction, both the caller and the callee can protect themselves from having the other's work run on their threads. On the one hand, a nonblocking-IO library may not want its callers to enqueue arbitrary work on the library's internal nonblocking-IO thread pool, at the risk of starving incoming work of unblocked threads to service incoming requests. On the other hand, a function that is running work on an inline executor may want to ensure that the library to which it is passing a future will definitely not run whatever work it has to do on the caller's thread. In either case, the extra executor adds cost even if it never runs any additional tasks. The extra executor would be unnecessary under the hypothesis of well-behaved code.

A simpler addition to `std::future`

We propose continuing to treat `std::future` as the vocabulary type for potentially asynchronous execution. We should not require that `std::future` make available any user-visible executor; we should minimise the set of cases where it is unclear on what executor work will run.

Instead, we propose modifying `std::future` to add a `.via()` method that takes an executor. `std::future::via` should consume the `std::future` and return a new future type.

This new future type is yet to be defined but should embody some of the same capabilities that are in `std::experimental::future` or `folly::Future`. In particular, it should add support for continuations using `.then`

methods, as most people expect. We will call this new future type `magic_future` here, in the knowledge that this name is not what we really want, to avoid bikeshedding about the naming here. `magic_future` should store its executor internally, such that it is well-defined to add an overload of `.then` that takes no executor. We would argue against adding any `.then` overloads that take an executor, because these overloads would lead to confusion about executor stickiness. Chaining calls to `.then` after calls to `.via` is just as readable and efficient: `someFuture.via(SomeExecutor{}).then(...)`. It is open to discussion whether this method should be restricted to r-value futures. We should additionally add a conversion, possibly implicit, from `magic_future` to `std::future`.

Therefore we might aim for something similar to:

```
template<class T>
class future {
    ...
    // Primary r-value via method
    template<class ExecutorT>
    std::magic_future<T> via(ExecutorT executor) &&;
    // Optional l-value via method
    template<class ExecutorT>
    std::magic_future<T> via(ExecutorT executor) const &;
};

template<class T>
class magic_future{
    ...
    // Implicit conversion to std::future
    operator std::future() &&;
    // r-value executor-less addition of continuation and return new future
    template<class FunctionT>
    magic_future<T> then(FunctionT task) &&;

    // Optional r-value then operation with executor and l-value then operations
    template<class ExecutorT, class FunctionT>
    magic_future<T> then(FunctionT task) const &;
    template<class ExecutorT, class FunctionT>
    magic_future<T> then(ExecutorT executor, FunctionT task) const &;
    template<class ExecutorT, class FunctionT>
    magic_future<T> then(ExecutorT executor, FunctionT task) &&;
};
```

In this world, `std::future` stays as the vocabulary type, with general day to day use unchanged. Our forwarding function as described above simplifies:

```
void myForwardingFunction() {
    auto future = LibraryA::getFromSomewhere(params);
    LibraryB::sendToSomewhere(future);
}
```

We no longer need to tell `LibraryA` what executor to complete its future on. `myForwardingFunction` does not need to know about executors at all. `LibraryA` did some work; `LibraryB` will do more work dependent on `LibraryA`'s work. The forwarder should not incur any cognitive load or runtime cost to construct an executor that exists purely to protect `LibraryA` from its callers.

As `std::future` will be carrying potentially unexecuted tasks, its core will likely have to carry a type-erased executor. This appears to be an implementation detail. Moreover, it is probably also safe to share the same core, with continuation support, between `std::future` and `std::magic_future` making the required set of conversion operations low-to-zero cost. We have implemented this in Folly by adding a `folly::SemiFuture` representing the continuation-free `std::future` and the original, continuation-enabled, `folly::Future` as a derived type having the functionality that we would expect of `magic_future`.

Templating the new future

If we continue to use `std::future` as the vocabulary type for APIs, we can consider templating our new `magic_future` on the executor type, both for efficiency and for interface precision. So our new future then becomes typed:

```
template<class T, class ExecutorT> class magic_future;
```

The executor-parameterized future type means we do not pass a future that supports continuations and yet has an unknown executor type, and hence an unknown set of capabilities, across library boundaries unless we explicitly do so with a polymorphic executor. This is important because it also means we do not pass a future that supports continuations and has an unknown forward progress guarantee for those continuations, as forward progress guarantees vary between executor types.

In the Concurrency TS design, we pass the completed future to the continuation. In Folly Futures, the primary interface is to pass a `folly::Try` type that wraps either the value or the exception with which the future was completed. Instead we should either pass a future type parameterized by the executor, or to simplify the argument list and to avoid implying the full set of future capabilities, optionally pass a separate executor to the continuation:

```
f.then([](ExecutorT e, auto result){/*...*/});
```

If the future is templated on the executor type we can use this information in the continuation. For example, if we want to enqueue work on the same executor as the current task is running on:

```
f.then([](ExecutorT e, auto value){e.execute([](ExecutorT e){/*...*/});});
```

With the precise type of the executor we can use the interface more flexibly - for example, by using knowledge about the structure of the executor type hierarchy:

```
f.then([](ThreadPoolThreadExecutor& e, auto value){
    doWorkA(value);
    ThreadPoolExecutor tpe = e.getParentPool();
    tpe.execute([value](ThreadPoolThreadExecutor e){doWorkB(value);});
});
```

In this case we know we are running on a member thread of a thread pool. We use this knowledge to get an executor representing the entire pool, or a strongly typed context from which we can get a member executor. We defer

knowledge of which thread ultimately runs the task to the runtime; once our task starts, we have a thread pool thread executor. Importantly for this example, the functions `doworkA` and `doworkB` run in the same thread pool, but may run in different threads within the single thread pool.

Note that we can default this type to be the polymorphic executor `magic_polymorphic_executor` (likewise, named so as to avoid bikeshedding over the name here, although likely based on the polymorphic wrappers proposed in P0443R2), which would provide us minimal information about the executor in the task. We may also allow converting a `std::magic_future<T, ExecutorT>` to a `std::magic_future<T, OtherExecutorT>` whenever `ExecutorT` is convertible to `OtherExecutorT`, and make all executors convertible to `magic_polymorphic_executor`.

We believe that by separating the two future types into the existing `std::future` extended with `std::future::via` and a new `magic_future`, rather than attempting drastically to widen the interface of `std::future`, we have much more flexibility in the design choices we can make.

Boost blocking and deferred execution

In p0679r0 Torvald Riegel expressed a desire to ensure that continuations and result extraction for futures provide boost-blocking guarantees. `folly::Future` and its executors do not provide this: we require a call to `.getVia` to ensure that a callback that has no currently known executor gets one, and chains of continuations with undriven executors will not execute.

In looking at whether we can produce a continuation-less version of `folly::Future` we saw a common case where a library wants to do some work on its own executor, and wants also to do some work on a caller-provided executor. For example, much of Facebook's networking library code will perform nonblocking-IO on an internal nonblocking-IO executor, but will deserialize messages on a caller-provided executor. This causes problems in practice where users find such libraries harder to learn, as it is not obvious at the call site what the purpose of the caller-provided executor is.

With good boost-blocking support we can avoid this. `std::future::get` should boost-block on the executor attached to the future. `std::future::via` similarly leads to boosting, but does so by ensuring that a task is added to the provided executor that drives, if necessary, the previously attached executor to ensure earlier tasks complete. In this way a whole chain of inline executors may be provided that drive each other in turn until the work is completed.

Assuming we have some deferred/manual executor type named `magic_deferred_executor` (same caveat about naming) that guarantees not to execute work immediately but to run it when the executor is driven later via the `.magic_drive` member function (same caveat about naming), we can ensure when we return a future from a library we can defer work until the caller calls `.get` or chains work through an executor of their choice. This means code like the following can be made to work:

```
std::future<T> LibraryA::getFromSomewhere(Params params) {
    magic_future tf = getRawNetworkData(params);
    return tf.via(magic_deferred_executor{}).then([](auto buffer){ return deserialize(buffer); });
}

int main() {
    auto f = getFromSomewhere(Params{});
    // Deserialization will happen some time after this point
```

```

    auto resultFuture = f.via(ThreadedExecutor{});
    // ...
    return 0;
}

```

This gives us control of what runs where, but with a simple, safe API for interacting between libraries. `f` then need not boost-block here, as that behaviour is a property of the executors, and any application of boost-blocking is thus defined by points at which executors are connected together - with the clarification that a call to `f.get()` is logically equivalent to `magic_deferred_executor e; auto f2 = f.via(e); e.magic_drive(); f2.get();`.

Boost-blocking of executors still has to be considered carefully, of course, to avoid recursive driving behaviour. We merely use a `magic_drive()` method as a potential interface for this that internals of futures would use.

A requirement arising from this is that any executor attached to a `std::future` should, in context, be boost-blocking at a minimum, or the work will never complete. For any user of a `std::future`, it is reasonable to expect that the future will complete eventually, but that the calling thread might have to do some additional work inline to achieve this.

Adding support for coroutines

A future that represents an asynchronous operation but provides only a synchronous `.get` operation is a reasonable design to interact with coroutines. Code that uses Folly Fibers, which is based on `boost::context`, appears synchronous in that it uses `.get()` on the future and the internal context switching is hidden behind the interface. Similarly, it is reasonable to extend the basic synchronous interface to the future to be awaitable and to work with `co_await`. In both these cases, information about the calling executor can be implicit in the calling context, either because it is really synchronous on a single executor in the case of a fiber or because the calling coroutine frame can carry information about where it is executing. We therefore are less likely to see issues with enqueueing a continuation onto an unexpected executor.

In summary

We argue that `std::future` should not be extended with continuations. It should remain a simple, wait-only type that serves a concrete purpose of synchronously waiting on potentially asynchronous work. We should extend `std::future` only to allow it to convert in the presence of an executor into a more sophisticated future type and to add the appropriate requirements for forward progress guarantees. This is extensible and flexible, and enables specialization based on the provided executor.